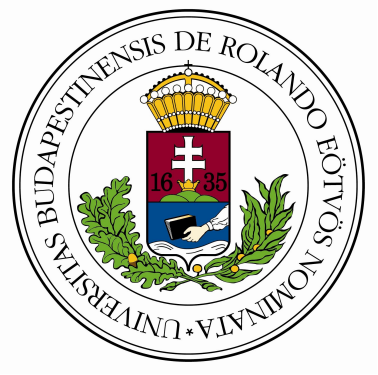
Konkurens programozás

Szálak között megosztott adatok



Kozsik Tamás

ELTE Eötvös Loránd Tudományegyetem

Java program memóriája

- Heap: objektumok
- Stack: alprogramhívások aktivációs rekordjai



Java program memóriája

- Heap: objektumok
- Stack: alprogramhívások aktivációs rekordjai
- Stack: alprogramhívások aktivációs rekordjai
- Stack: alprogramhívások aktivációs rekordjai
- Stack: alprogramhívások aktivációs rekordjai
- ...



Végrehajtási verem

(execution stack)

- Aktivációs rekordok
  - activation record
  - stack frame
- Paraméterátadás
- Lokális változók
- Kivételek terjedése



Példa

```
class MainThread {
    public static void main( String[] args ){
        // method calls may come here
        // ... call chain is administered on the stack
        new OtherThread().start();    // new stack is created
        // method calls may come here
        // independent execution from other thread
    }
}

class OtherThread extends Thread {
    @Override public void run(){
        // method calls may come here
        // ... call chain is administered on the stack
    }
}
```



Szál befejeződése

Nem függ az elindított szálaktól!



Program befejeződése

- Minden szál befejeződött



Program befejeződése

- Minden szál befejeződött
- Legfeljebb *daemon*-szálak futnak már csak

```
package java.lang;
public class Thread implements Runnable {
    public final boolean isDaemon() ...
    public final void setDaemon(boolean on) ...
    ...
}
```



Program befejeződése

- Minden szál befejeződött
- Legfeljebb *daemon*-szálak futnak már csak

```
package java.lang;
public class Thread implements Runnable {
    public final boolean isDaemon() ...
    public final void setDaemon(boolean on) ...
    ...
}
```

- Virtuális gép leállítása



Kivétel fellépése

```
class MainThread {
    public static void main( String[] args ){
        new OtherThread().start();
        throw new RuntimeException();
    }
}

class OtherThread extends Thread {
    @Override public void run(){
        ...
    }
}
```



Lehetséges, hogy a kivétel csendben lép fel

```
class MainThread {
    public static void main( String[] args ){
        new OtherThread().start();
        ...
    }
}

class OtherThread extends Thread {
    @Override public void run(){
        throw new RuntimeException();
    }
}
```



Segíthet...

```
class MainThread {
    public static void main( String[] args ){
        new OtherThread().start();
        ...
    }
}

class OtherThread extends Thread {
    @Override public void run(){
        try {
            throw new RuntimeException();
        } catch( Throwable t ){
            t.printStackTrace();
        }
    }
}
```



Közösen használt adatok?

- Stacken tárolt adatok csak egy szálból
- Heapen tárolt adatok: szinkronizáció



Adat: változó

- Mi a típusa?
  - `boolean`
  - `char`
  - `byte`
  - `short`
  - `int`
  - `long`
  - `float`
  - `double`
  - referencia



Adat: változó

- Mi a típusa?
  - `boolean`
  - `char`
  - `byte`
  - `short`
  - `int`
  - `long`
  - `float`
  - `double`
  - referencia
- Hol tárolódik?
  - Stacken
  - Heapen



Programvégrehajtás

- Változó beolvasása a „memóriából”
- Változó kiírása a „memóriába”
- Számítási lépés
- Vezérlésátadás



Programvégrehajtás

- Változó beolvasása a „memóriából”
- Változó kiírása a „memóriába”
- Számítási lépés
- Vezérlésátadás

- Szinkronizáció
- Szálak indulása, leállása, bevárása...

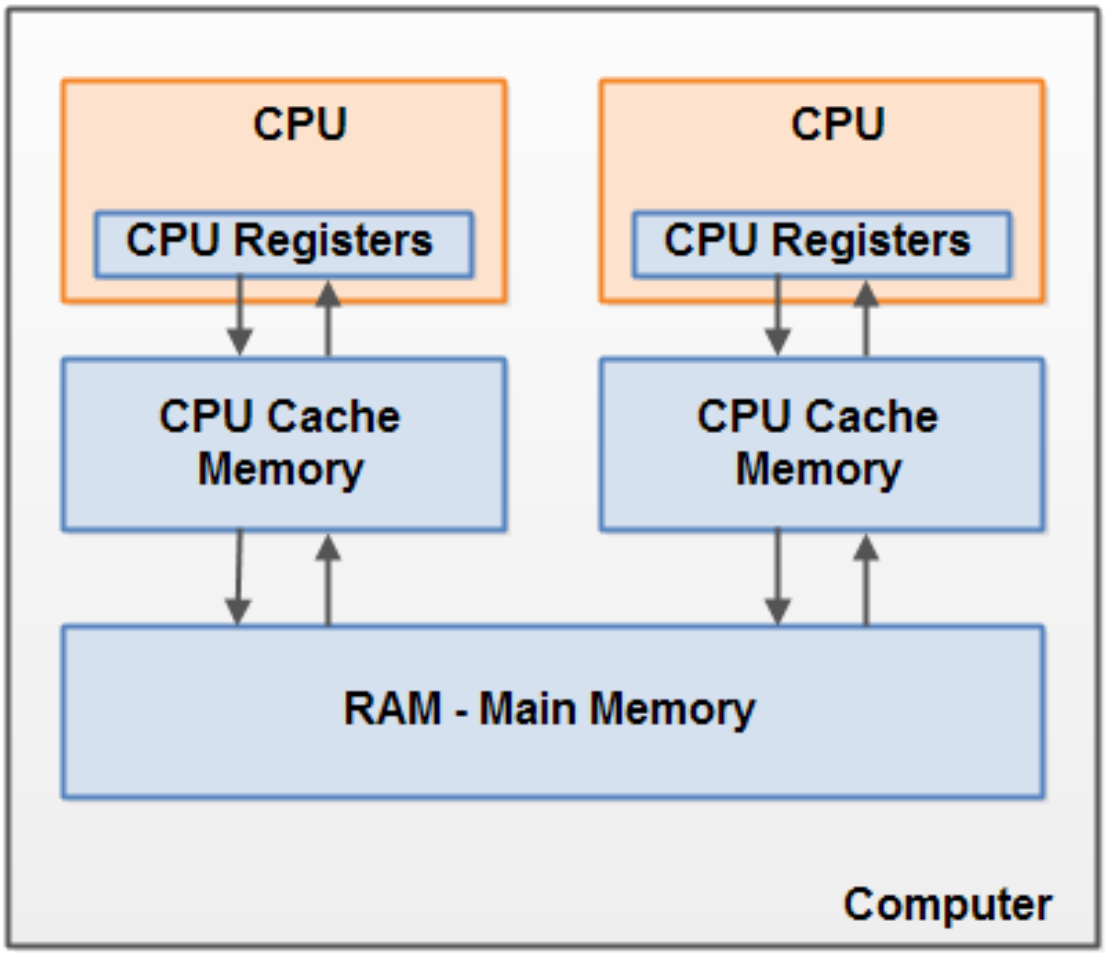


Programvégrehajtás: ami fontos most nekünk

- **Változó beolvasása a heap „memóriából”**
- **Változó kiírása a heap „memóriába”**
- Változó beolvasása a stack „memóriából”
- Változó kiírása a stack „memóriába”
- Számítási lépés
- Vezérlésátadás
- **Szinkronizáció**
- **Szálak indulása, leállása, bevárása...**



Memória


CPU
CPU Registers
CPU
CPU Registers
CPU Cache Memory
CPU Cache Memory
RAM - Main Memory
Computer

Memóriaműveletek emberi skálán

órajel	0.4 ns	1 sec
L1 cache	0.9 ns	2 sec
L2 cache	2.8 ns	7 sec
L3 cache	28 ns	1 min
DDR memória	~100 ns	4 min
SSD I/O	50-150 microsec	1,5-4 nap
HDD I/O	1-10 ms	1-9 hónap
Internet	65 ms	5-10 év



Mi lehet megosztva a szálak között?

```
class Foo {
    int n = 4;
    java.util.List<Foo> others = new java.util.ArrayList<>();
    public void add( int on ){
        Foo other = new Foo();
        other.n = on;
        others.add(other);
    }
    public static void main( String[] args ){
        Foo foo = new Foo(); foo.add(5);
        ...
    }
}
```



Mi lehet megosztva a szálak között?

```
class Foo {
    int n = 4;
    java.util.List<Foo> others = new java.util.ArrayList<>();
    public void add( int on ){
        Foo other = new Foo(); other.n = on; others.add(other);
    }
    public static void main( String[] args ){
        Foo foo = new Foo(); foo.add(5);
        new MyThread(foo).start(); ...
    }
}
class MyThread extends Thread {
    Foo foo;
    MyThread( Foo foo ){ this.foo = foo; }
    @Override public void run(){ ... }
}
```



Egy szálra korlátozott hozzáférés

(confined to a thread)

- Stacken tárolt változó!
- Heapen tárolt változó?
  - Nincs nyelvi támogatás erre Javában
  - A programozó felelőssége



Heapen tárolt adat is lehet egy szálra korlátozott

```
public String boring( String word, int num ){
    StringBuilder builder = new StringBuilder();
    for( int i=0; i<num; ++i ){
        builder.append(word);
        builder.append(' ');
    }
    builder.deleteCharAt(builder.length()-1);
    return builder.toString();
}
```



Szálra korlátozott objektum a heapen

```
class Foo {
    int n = 4;
    java.util.List<Foo> others = new java.util.ArrayList<>();
    public void add( int on ){
        Foo other = new Foo(); other.n = on;  // thread-confined
    }
    public static void main( String[] args ){
        Foo foo = new Foo(); foo.add(5);
        new MyThread(foo).start(); ...
    }
}
class MyThread extends Thread {
    Foo foo;
    MyThread( Foo foo ){ this.foo = foo; }
    @Override public void run(){ ... }
}
```



Becsapós példa

```
class Foo {
    int n = 4;
    java.util.List<Foo> others = new java.util.ArrayList<>();
    public void add( int on ){
        Foo other = new Foo(); other.n = on; others.add(other);
    }
    public static void main( String[] args ){
        final Foo foo = new Foo(); foo.add(5);
        new Thread( () -> foo.add(9) ).start();
        ...
    }
}
```



A `final` kulcsszó a Javában

- Osztály: nem lehet származtatni belőle
- Metódus: nem lehet felüldefiniálni
- Változó:
  - Nem lehet módosítani az értékét
  - Lehet nemlokális változója beágyazott osztálynak
  - Nem igényel szinkronizációt az olvasása (pontosítandó!)



Closure

```
interface Valued {
    int value();
}
class Bar {
    static Valued create( int bound ){
        final int num = (int)(bound * Math.random());
        return new Valued(){   public int value(){
                                    return num;
                               }
                           };
    }
    public static void main( String[] args ){
        Valued valued = create(100);
        System.out.println( valued.value() );
    }
}
```



Closure

```
interface Valued {

    int value();

    static Valued create( int bound ){
        int num = (int)(bound * Math.random());
        return () -> num;
    }

    public static void main( String[] args ){
        Valued valued = create(100);
        System.out.println( valued.value() );
    }
}
```



Closure szálban

```
class Foo {
    public static void main( String[] args ){
        final int num = (int)(100 * Math.random());
        new Thread( () -> {
                            for( int i=0; i<num; ++i ){
                                System.out.println( i );
                            }
                        }
                  ).start();
        for( int i=0; i<num; ++i ){
            System.out.println( i );
        }
    }
}
```



Nem jól szinkronizált program

```
class Foo {
    public static void main( String[] args ){
        final int num = (int)(100 * Math.random());
        final ArrayList<Integer> list = new ArrayList<>();
        new Thread( () -> {
                                for( int i=0; i<num; ++i ){
                                    list.add( i );
                                }
                            }
                    ).start();
        for( int i=0; i<num; ++i ){
            list.add( i );
        }
    }
}
```



A `final` változók

- Nem lehet módosítani az értékét
- Lehet nemlokális változója beágyazott osztálynak
- Nem igényel szinkronizációt az olvasása (pontosítandó!)



A `final` referenciák

- Nem lehet módosítani az értékét
  - A referencia módosíthatatlan
  - Nem állítható át másik objektumra
  - A hivatkozott objektum módosítható lehet!



A `final` referenciák

- Nem lehet módosítani az értékét
  - A referencia módosíthatatlan
  - Nem állítható át másik objektumra
  - A hivatkozott objektum módosítható lehet!
- Lehet nemlokális változója beágyazott osztálynak
  - Pl. egy szál programjának
  - A nonlokális referencia lemásolódik a szál vermébe
  - A hivatkozott objektumon a heapen van, azaz *megosztott*



A `final` referenciák

- Nem lehet módosítani az értékét
  - A referencia módosíthatatlan
  - Nem állítható át másik objektumra
  - A hivatkozott objektum módosítható lehet!
- Lehet nemlokális változója beágyazott osztálynak
  - Pl. egy szál programjának
  - A nonlokális referencia lemásolódik a szál vermébe
  - A hivatkozott objektumon a heapen van, azaz *megosztott*
- Nem igényel szinkronizációt az olvasása (pontosítandó!)
  - A referencia elérése nem igényel szinkronizációt
  - A hivatkozott objektum manipulálása szinkronizációt igényel!



A `final` szerepe többszálú programban

Több szál által közösen használt `final` változók (adattagok) elérése nem igényel szinkronizációt.

(Tanulunk majd később erre egy feltételt, ami kell még!)



Alapelvek

- A helyes működést nem áldozzuk fel
- Az egyszerűséget és karbantarthatóságot csak okkal áldozzuk fel
  - Nem optimalizálunk idő előtt
  - Egymásnak ellentmondó szempontok
  - Profilozás



Helyesség

Hibás az a program, amely szinkronizáció nélkül oszt meg módosuló állapotot szálak között.

- Ne osszunk meg a változót szálak között
- A változó legyen módosíthatatlan
- Használjunk szinkronizációt



Módosíthatatlan (immutable) adatok

- Ha az objektum mezői `final`-ek
- Funkcionális stílusú programozás
- Új érték új objektumban jön létre
- Szekvenciális programban: overhead



Módosíthatatlan (immutable) adatok

- Ha az objektum mezői `final`-ek
- Funkcionális stílusú programozás
- Új érték új objektumban jön létre
- Szekvenciális programban: overhead

- Konkurens programban: nyereség
- Nem kell szinkronizációt használni



Nincsenek megosztott adatok

```
class Foo {
    public static void main( String[] args ){
        final int num = (int)(100 * Math.random());
        new Thread( () -> {
                            for( int i=0; i<num; ++i ){
                                System.out.println( i );
                            }
                        }
                  ).start();
        for( int i=0; i<num; ++i ){
            System.out.println( i );
        }
    }
}
```



Csak módosíthatatlan adatot osztunk meg

```
class Foo {
    public static void main( String[] args ){
        final Integer num = (int)(100 * Math.random());
        new Thread( () -> {
                                for( int i=0; i<num; ++i ){
                                    System.out.println( i );
                                }
                            }
                    ).start();
        for( int i=0; i<num; ++i ){
            System.out.println( i );
        }
    }
}
```



Módosuló adat megosztása szálak között: szinkronizáció!

```
public class BankServer {
    public static void main( String[] args ){
        final BankData data = BankData.initialize(args);
        while( true ){
            try {
                Connection c = Connection.accept();
                ClientHandler ch = new ClientHandler(c,data);
                new Thread(ch).start();
            } catch( java.io.IOException e ){ ... }
        }
    }
}

class ClientHandler implements Runnable { ... }

class BankData { /* szinkronizációval védett */ }
```



Kölcsönös kizárással védett adatok

```
public class Account {
    private int balance;
    public synchronized int balance(){
        return balance;
    }
    public synchronized void deposit( int amount ){
        assert amount > 0;
        balance += amount;
    }
    public synchronized void withdraw( int amount )
            throws InsufficientBalanceException {
        assert amount > 0;
        if( balance >= amount ) balance -= amount;
        else throw new InsufficientBalanceException();
    }
}
```



Többszálú használatra alkalmas adatszerkezettel

```
public class BankData {
    public static BankData initialize( String[] args ){
        return new BankData(args);
    }

    private final List<Account> accounts;

    public BankData( String[] args ){
        ...
        accounts = Collections.synchronizedList(...);
    }

    public void deposit( int amount, int accountId ){
        accounts.get(accountId).deposit(amount);
    }
    ...
```



Holtpontmentes

```
public class BankData {
    ...
    public boolean transfer( int amount, int fromId, int toId ){
        Account f = accounts.get(fromId), t = accounts.get(toId);
        Account fst = (fromId < toId) ? f : t,
                snd = (fromId < toId) ? t : f;
        synchronized(fst){ synchronized(snd) {
            try {
                f.withdraw(amount); t.deposit(amount);
                return true;
            } catch( InsufficientBalanceException e ){
                return false;
            }
        }}
    }
}
```

